SpaceFusion: Advanced Deep Learning Operator Fusion via *Space-Mapping Graph*

Liang Zhu
Shanghai Jiao Tong University
Shanghai, China
liang.z@sjtu.edu.cn

Jianguo Yao*
Shanghai Jiao Tong University
Shanghai, China
jianguo.yao@sjtu.edu.cn

Haibing Guan
Shanghai Jiao Tong University
Shanghai, China
hbguan@sjtu.edu.cn

Abstract

This work proposes *SpaceFusion*, an advanced scheduler for efficient deep learning operator fusion. First, we develop a novel abstraction, the *Space-Mapping Graph* (SMG), to holistically model the spatial information of both inter- and intra-operator dependencies. Subsequently, we introduce the spatial and temporal slicers to decompose the fused spaces defined in SMGs, generating fusion schedules by analyzing and transforming dependencies. Finally, we present auto-scheduling methods that use the slicers to automatically create high-performance fusion schedules tailored to specific hardware resource configurations. End-to-end performance evaluations reveal that SpaceFusion achieves up to 8.79x speedup (3.54x on average) over baseline implementations from Huggingface for Transformer models, and a maximum of 2.21x speedup compared to the state-of-the-art manually-tuned implementations powered by FlashAttention.

CCS Concepts: • **Computing methodologies** → **Machine learning**; **Parallel computing methodologies**.

Keywords: Machine Learning, Operator Fusion, Deep Learning Compilation,

ACM Reference Format:
Liang Zhu, Jianguo Yao, and Haibing Guan. 2025. SpaceFusion: Advanced Deep Learning Operator Fusion via *Space-Mapping Graph* . In *Twentieth European Conference on Computer Systems (EuroSys '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3689031.3696087

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EuroSys '25, March 30–April 3, 2025, Rotterdam, Netherlands
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1196-1/25/03…$15.00
https://doi.org/10.1145/3689031.3696087

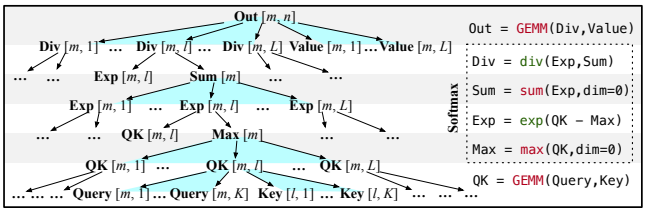


Figure 1. Complex Data Dependencies in MHA. Left: the dependency graph of a single element in the output tensor. Right: the simplified computational definitions.

1 Introduction

Tensor computations defined by deep learning models manifest two trends: increasing computational scale and heightened computational complexity[27, 28, 37, 40, 51, 52], making it more expensive and challenging to deploy them as efficient, rapid-response inference services, whether on cloud infrastructures or local devices[43, 44]. Increasing computational scale necessitates the allocation of more memory resources for larger model weights, input data, output data, and intermediate data, resulting in higher memory I/O overhead and increased memory usage. Higher computational complexity, primarily arising from more non-element-wise operators such as Generalized Matrix Multiplication (GEMM) and Softmax[5], escalates the number of computation cycles and introduces more intricate data dependencies. Operator fusion techniques (also known as kernel fusion or layer fusion), mitigate the issue brought by larger computational scale via caching intermediate computation results in storage mediums closer to the compute units, enabling their on-chip reuse. This enables faster responses for inference services while reducing memory costs. However, operator fusion faces challenges in fusing non-element-wise operators[43, 70].

One of the main reasons is the tradeoff between inter-block dependency-free parallelism and intra-block data locality[61]. In a typical GPU architecture featuring a memory hierarchy[41], different thread blocks are restricted to exchange data through global memory (off-chip, slower, greater in capacity, visible to all thread blocks). A fusion aiming to cache intermediate results in shared memory (on-chip, faster, smaller in capacity, visible within a thread block) requires no dependency between scheduled thread blocks for parallelization in the

GPU architecture[26]. For example, the Multi-Head Attention (MHA)[58] contains three non-element-wise operators, including two GEMMs and a Softmax. Figure 1 illustrates the dependencies from a single element of the output tensor to the other tensor elements. This complex dependency pattern exhibits (1) deeply nested dependencies between elements, layer by layer, and (2) wide dependency ranges covering the whole range of a tensor dimension. As a result, scheduling fused MHA into thread blocks while naively respecting the wide and nested dependency pattern leads to low intra-block data locality, resulting in intra-block inefficiency or even fusion failures.

Existing approaches to operator fusion are classified into two *manually-tuned fusion* and *auto-tuned fusion*. Manually-tuned fusion[11, 22, 30, 31, 65] is the dominant approach to fusing non-element-wise operators. These methods target specific tensor computations. Specific dataflow and parallelism strategies are cleverly designed by domain experts to achieve fusions with complex dependencies. Unfortunately, manually-tuned fusion suffers from two key limitations: limited generality and high development cost.

Auto-tuned fusion provides a promising alternative for its generality and cost-effectiveness. However, fusing non-element-wise operators is beyond the capabilities of most existing deep learning (DL) frameworks and compilation systems[4, 6, 29, 34, 43, 57, 66, 67]. The majority of them[6, 29, 34, 43, 67] rely on high-level abstractions, i.e., graph-based abstractions like dataflow graph (DFG), targeting element-wise operator fusion. These abstractions fail to provide the abstraction of intra-operator dependencies and hence are incapable of tackling complex dependencies defined by operators. Other approaches[4, 57, 66], using low-level abstractions such as polyhedral models, also face limitations in efficiently identifying optimal operator fusion solutions due to the vast search space defined by fine-grained abstractions. Recent works[49, 68, 70] have achieved breakthroughs in fusions of a broader range of operators, but problems remain. AStitch[70] uses rule-based approaches to fuse memory-intensive (MI) operators. Chimera[68] devises a purely analysis-based approach to fuse compute-intensive (CI) operators. Neither addresses the fusion of both MI and CI operators. Welder[49] leverages the *tile-graph* abstraction for finer-grained schedules (tile-level) of inter-operator intermediate tensors within the memory hierarchy. However, the inability to abstract intra-operator dependencies limits its capacity to construct a holistic optimization space.

In this work, we present *SpaceFusion*, an advanced scheduler for efficient operator fusion. The key design is a spatial abstraction termed *Space-Mapping Graph* (SMG) (Section 4.1). Despite being a lightweight graph-based approach, the SMG abstraction effectively captures complex dependencies in multiple operators, enabling the construction of a holistic optimization space. Then we devise two slicing methods to schedule SMGs. The *spatial slicer* (Section 4.2) slices an SMG into multiple parallel-executed SMG blocks, each independent of the others. The *temporal slicer* (Section 4.3) slices an SMG block into multiple serially-executed intra-blocks to reduce the on-chip memory footprint, and to exploit the possible optimization opportunities. Finally, we design *auto-scheduling* methods (Section 5) for SMG. The *resource-aware slicing* method (Section 5.1) leverages spatial/temporal slicers to automate the generation of fusion schedules that adhere to hardware resource constraints. The *SMG partition* method (Section 5.2) partitions unschedulable SMGs due to hardware resource constraints into smaller SMGs and resubmits them to the slicing method. We identify the optimal schedule within the generated search space.

Evaluations reveal SpaceFusion's ability to exploit hidden fusion opportunities in intricate dependencies and deliver performance benefits across diverse GPU architectures. For subgraph performance, SpaceFusion achieves a maximum speedup of 10.35x (with an average of 5.31x) over manually-tuned unfused baseline, and up to 4.03x speedup compared to manually-tuned fused libraries. For end-to-end performance, SpaceFusion achieves a maximum speedup of 8.79x (with an average of 3.54x) to baseline implementations by Huggingface[62] for Transformer models, and up to 2.21x speedup to the state-of-the-art (SOTA) manually-tuned implementation[31] powered by FlashAttention[22].

2 Background and Prior Knowledge

This section discusses the background and prerequisite knowledge of operators for tensor computation in DL models. Further, we explore the complex dependency patterns in the tensor computations defined by an individual operator and multiple operators.

Operators for tensor computation encompass element-wise and non-element-wise operators. Element-wise operators, characterized by straightforward dependency patterns, can be simply fused with other operators, e.g., by in-place inlining[6]. Non-element-wise operators, such as GEMM, Softmax, ReduceMean, and element-wise with broadcast, constitute the primary source of complex dependencies[43].

Dependencies in a single operator. Dependencies in most non-element-wise operators can be further broken down. For example in GEMM, given a matrix multiplication $C_{M\times N} = A_{M\times K}B_{N\times K}$, a single element of the output tensor is defined as $c_{mn} = \sum_{k=1}^{K} a_{mk}b_{nk}$, where a, b and c are the elements in tensor A, B and C, respectively. For each $n \in [1, N]$, a single element a_{mk} is used N times for computing the intermediate elements $a_{mk}b_{1k}$, $a_{mk}b_{2k}$, ..., and $a_{mk}b_{Nk}$, which means one input element a_{mk} is required by N intermediate elements along the whole range of N. We distinguish this dependency pattern as *One-to-All*. A similar One-to-All dependency exists between b_{nk} and the intermediate elements for each $m \in [1, M]$. Then, an output element c_{mn} is calculated via summing K intermediate elements $a_{m1}b_{n1}$, $a_{m2}b_{n2}$,

Table 1. Decoupled Dependencies in Representative Operators. ◯, △, and × denote the presence, potential presence, and absence of the dependencies, respectively.

Representative Operators	One-to-One	One-to-All	All-to-One
GEMM	×	◯	◯
Einsum	△	△	△
Softmax, LayerNorm, BatchNorm	◯	◯	◯
ReduceMax, ReduceMean	×	×	◯
Element-wise w/ broadcast	◯	◯	×

Table 2. A Comparative Analysis of Representative Works for Operator Fusion. Abstraction capabilities for fusion (perceiving inter-operator and intra-operator dependencies), and fusion schedule capabilities (memory access transformation, dependency transformation, memory hierarchy scheduling, and hardware resource awareness) are listed.

Year	Work	Abstraction Capability		Schedule Capability			
		Inter-op Dep	Intra-op Dep	Mem Trans	Dep Trans	Mem Hier	HW Rsrc
2020	Ansor[67]	Tensor-wise	×	◯	×	×	×
2021	DNNFusion[43]	Tensor-wise	×	◯	×	×	×
2022	AStitch[70]	Tensor-wise	×	◯	×	◯	◯
2023	Welder[49]	Tile-wise	×	◯	×	◯	◯
2024	**SpaceFusion**	Space-wise	◯	◯	◯	◯	◯

..., and $a_{mK}b_{nK}$ along the whole range of K. We distinguish this pattern as *All-to-One*. Therefore, the dependencies in GEMM can be further decoupled into One-to-All and All-to-One dependencies. Table 1 shows decoupled dependencies may exist in representative non-element-wise operators, where *One-to-One* indicates element-wise dependencies.

Complex dependencies in multiple operators. Dependencies in multiple operators are more complex, for the complex dependencies that appear in each operator are nested and combined, always forming a wide and deep dependency tree, as Figure 1 corroborates. Given a simplified computational definition of MHA shown on the right side of Figure 1, a single element of the output tensor depends directly or indirectly on $(2LK + 4K + 2)$ elements from 8 tensors with 6 layers nested dependencies containing 6 One-to-Alls and 4 All-to-Ones, where L and K indicate sequence length and feature dimension, respectively. This kind of complex dependency pattern exhibits (1) deeply nested dependencies between elements, layer by layer, and (2) wide dependency ranges covering the whole range of a tensor dimension.

3 Major Challenges

We argue two challenges in generating efficient schedules for fusing operators with complex dependencies discussed above. Table 2 compares representative works on operator fusion to present methodology differences for dealing with fusion problems and illustrate the challenges comprehensively.

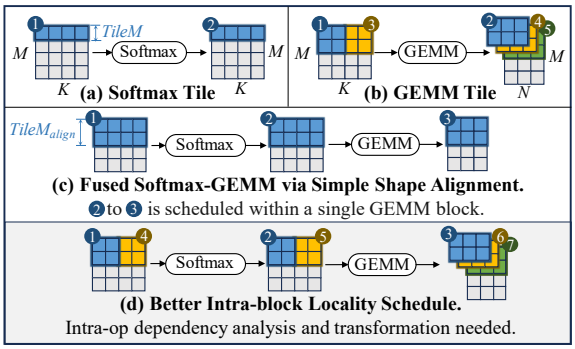


Figure 2. An example of Softmax-GEMM fusion. (a) and (b) depict input-output tile mappings in Softmax and GEMM. (c) is a fusion via shape alignment of the intermediate tiles. (d) is a fusion with better locality. Serial numbers in each sub-figure indicate the order in which tiles were accessed.

Challenge 1: *Identifying a suitable abstraction granularity that is both effective and efficient for operator fusion problems.* There are two main levels of abstraction for tensor computations, *high-level* and *low-level abstractions*. High-level abstractions, i.e., graph-based abstractions such as dataflow graphs (DFGs), represent tensor computations at the operator level, where nodes denote operators and edges denote tensor-wise inter-operator dataflow dependencies. Most DL frameworks and compilers[1, 6, 7, 29, 34, 43, 45, 67] fuse operators via high-level abstractions at the graph optimization stage of compilation, enabling easy fusion decisions through rule-based methods. However, these abstractions, constrained by the *dependency barriers* between operators, where intra-operator dependencies are inaccessible at the inter-operator level, are *ineffective* in handling situations involving complex dependencies within operators.

Low-level abstractions work in a finer granularity, such as Halide-based intermediate representation (IR)[47] with the scheduling languages (at the function/loop level), and the polyhedral model[25] (at the statement/iteration level). These abstractions eliminate operator dependency barriers through loop-level primitives, such as loop fusion. However, low-level abstractions are overly fine-grained for operator fusion problems. They capture irrelevant or redundant information for fusion problems. Furthermore, the solution space grows with the number of statements and the size of iteration domains in tensor programs. These problems render low-level abstractions *inefficient* for solving the operator fusion problem, hindering the design and implementation of methodologies for the optimal solution. Therefore, we need to identify an appropriate abstraction granularity between high-level and low-level abstractions, such that it is *both effective and efficient* in solving the operator fusion problem.

Challenge 2: *To devise an abstraction method to capture, analyze, and schedule dependencies in operators holistically.*

Recent compilers employ *tile-level abstractions* for fusion. The tile-graph in Welder[49] refines operator dependencies to tile-wise granularity and stitches intermediate tiles via shape alignment, allowing precise scheduling within the memory hierarchy. However, the intra-operator dependencies remain unabstracted, replaced by the shape mappings from input to output tiles in an operator. Note that this compressed information discards the original dependencies. Figures 2 (a) and (b) illustrate the tile shape mappings between the input and output tiles of Softmax and GEMM. Stemming from global reductions along the K axis, each output element in Softmax $out[m,k]$ incorporates all input elements within its corresponding row $in[m,0{:}K]$. Figure 2 (c) illustrates the shape alignment of the output tile of Softmax with the input tile of GEMM. This fusion schedules the GEMM tiles that could have been executed serially to be computed in a larger single GEMM block, resulting in a poor intra-block locality in Softmax and GEMM (e.g. the 16x256 Softmax block and the 16x64x256 GEMM block for $TileM_{align}$=16, K=256 and N=64), and even fusion failures (e.g. for K=1024, the 16x1024 intermediate tiles may not fit in the limited shared memory).

Figure 2 (d) illustrates a fusion schedule with enhanced intra-block locality, featuring a new tile order and superior tile shapes (e.g. a 64x64x64 GEMM block) with memory overlapping among tiles of different colors (e.g., ❶ and ❹, ❷ and ❺), facilitating the Softmax-GEMM fusion for larger K values. Generating this schedule for the desired tile order requires *holistic* dependency abstraction, analysis, and transformation of the reductions in both Softmax and GEMM. Therefore, it requires an abstraction method that encompasses *multiple operators* in an optimization space, capturing and analyzing *both intra- and inter-operator* dependencies, and transforming them as needed to pursue a globally optimal solution.

4 Abstraction Methods

In this section, we introduce a graph-based abstraction termed *Space-Mapping Graph* (SMG). The following subsections provide a detailed introduction to SMG and explain how to slice SMGs for scheduling parallelization and serialization.

4.1 Space-Mapping Graph

A Space-Mapping Graph (SMG) retains the basic structure of graph-based abstraction, where the *Computational Spaces* are the nodes of the graph and the *Space Mappings* are directed edges connecting the nodes.

Computational Spaces are conceptualized into two distinct categories: (1) *Data spaces* abstract tensors, including input tensors, output tensors, intermediate tensors, and weight tensors employed in tensor computations; (2) *Iteration spaces* model the nested loop structures of defined computations. For example, Figure 3 shows the computational definition of matrix multiplication and its corresponding SMG. As is shown in Figure 3 (c), this SMG comprises four nodes, with

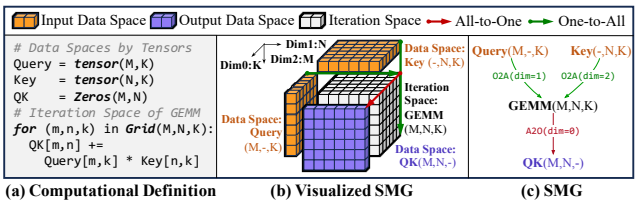


Figure 3. The SMG for a Single Operator GEMM. (a) the definition of $QK = GEMM(Query, Key)$, (b) the visualized SMG for GEMM, and (c) the SMG for GEMM are shown.

the orange and purple nodes representing data spaces that correspond to the $Query$, Key, and QK tensors mentioned in Figure 3 (a). The node $GEMM$ in black is the iteration space defined by the loop structure in Figure 3 (a).

Space Mappings abstract One-to-One, One-to-All, and All-to-One mapping relations[1] between computational spaces as is discussed in Section 2, possessing their own geometric directions in the SMG. *Intra-operator mappings* define relations between data and iteration spaces within a single operator, effectively encapsulating read and write dependencies. *Inter-operator mappings* connect one operator's output to another's input, modeling the dataflow between them. Inter-operator mappings are always One-to-One. For example again in Figure 3 (c), the directed edges in green represent the One-to-All mapping. The One-to-All mapping $Query \rightarrow GEMM$ denotes that data space $Query$ is reused by iteration space $GEMM$ along the second dimension, thus, possessing a distinct direction vector along the second dimension (dim=1). The red-directed edge is an All-to-One mapping $GEMM \rightarrow QK$, denoting the ReduceSum in iteration space $GEMM$, with the direction vector along the first dimension (dim=0). The visualized SMG in Figure 3 (b) better demonstrates the geometric space concept of mappings.

Compared to traditional DFGs, SMG introduces three key differences: (1) SMG incorporates dimensional information into nodes, constructing them as geometric spaces. This is fundamental to SMG's ability to preserve critical information. (2) SMG abstracts the iteration space. For an operator, the iteration spaces act as intermediate nodes, decoupling direct dependencies between input and output nodes into multiple indirect dependency mappings. This design is particularly suited for representing operators where the input and output spaces are not in One-to-One mappings. (3) SMG categorizes the decoupled dependency patterns into three types of space mappings. These space mappings enhance SMG's expressiveness in handling complex dependencies and provide the necessary information for holistic analysis and scheduling of dependencies.

[1]This work focuses on globally-ranged mappings as discussed in Section 2. Fusions of partially-ranged mappings, such as 2D convolution fusion[49, 60], are not being discussed here.

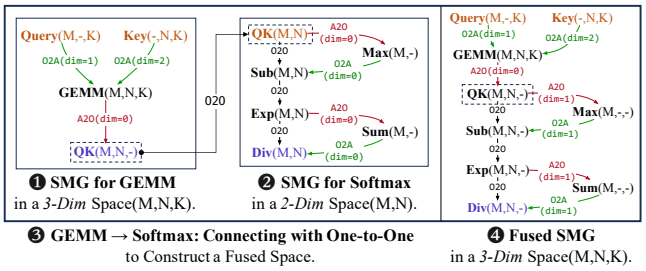


Figure 4. Connecting SMGs to Construct a New SMG for the Fused Computational Space.

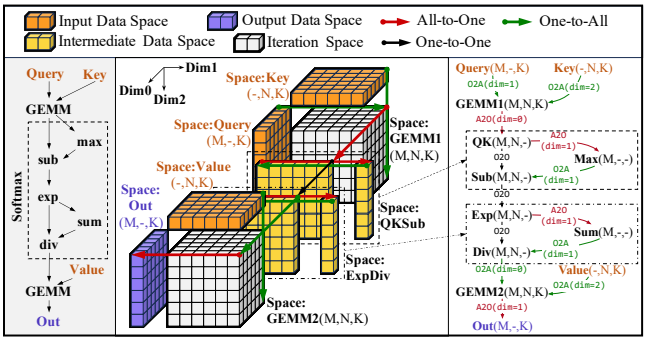


Figure 5. An SMG Example for Multi-Head Attention.

Building a fused SMG via connecting SMGs of single operators with intermediate data space dimension alignment: Figure 4 depicts the process of constructing a fused space for two individual operators, GEMM and Softmax, resulting in a fused SMG. ❶ SMG for GEMM defines a 3-dim space. ❷ SMG for Softmax defines a 2-dim space. ❸ GEMM's output data space QK (in purple) and Softmax's input data space QK (in orange) are connected with a One-to-One mapping. ❹ A fused SMG is constructed via fusing $QK(M,N,-) \rightarrow QK(M,N)$ into a single intermediate data space $QK(M,N,-)$ with dimension alignment, where "−" is a dimension placeholder for K in GEMM. The constructed SMG defines a new 3-dim space. Employing this method, we can establish a multi-operator fusion space wherein One-to-One mappings construct the main dataflow across operators.

An illustrative example of SMG. In Figure 5, (a) DFG of the simplified MHA, (b) visualized SMG, and (c) the corresponding SMG are presented. The computation of MHA takes place within a five-dimensional fused computational space (BatchDim, HeadDim, Dim2, Dim1, Dim0). We focus our discussion on the three-dimensional subspace[2] composed of the final three dimensions (Dim2, Dim1, Dim0). The visualized SMG fuses some of the spaces connected with One-to-One mappings for visual simplicity, as is depicted in dotted

[2]BatchDim and HeadDim are not involved to enhance the visual representation. Both are dimensions without dependencies on the others.

Table 3. Slicer Applications for Mappings in the Dimension. ◯ and × respectively denote the application and non-application of slicers, while △ signifies the necessity for further analysis to determine whether slicing is warranted.

Mappings in the Dimension	Spatial Slicer	Temporal Slicer
None	◯	◯
One-to-One	×	×
Input One-to-All	◯	◯
Other One-to-All	×	◯
Independent All-to-One(s)	×	◯
Dependent All-to-Ones	×	△

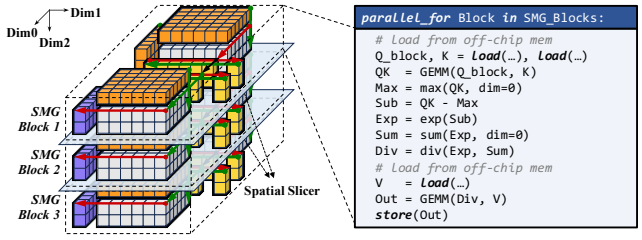


Figure 6. Spatial Slicers Applied along Dim2.

boxes in Figure 5 (b) and (c). As is discussed in Section 2, a MHA computation involves three non-element-wise operators with 6 One-to-Alls and 4 All-to-Ones. The visualized SMG in Figure 5 (b) depicts these 10 mappings and their geometric position information. The 4 red arrows represent the 4 All-to-Ones resulting from the reduction operations in MHA. The first originates from GEMM1, the second and third originate from Softmax, and the fourth originates from GEMM2. It is evident that the last three of these four All-to-Ones are geometrically parallel, with their directions aligned along Dim1, while the first All-to-One from GEMM1, towards Dim0, is orthogonal to the last three All-to-Ones.

4.2 Spatial Slicer: Exploiting SMG Parallelization

A *spatial slicer* slices an SMG along given dimensions into multiple independent, parallelizable *SMG blocks*. Each SMG block is to be mapped to a thread block in GPUs. To prevent dataflow dependencies between SMG blocks generated after spatial slicing, the spatial slicer generally avoids orthogonal mappings. Table 3 encapsulates the varied decisions made when applying slicers to slice an SMG along given dimensions with the presence of diverse mappings. Specifically, spatial slicers do not slice any mappings except for the input One-to-All. The source of an input One-to-All is an input data space abstracting the input of a kernel function, which is stored in global memory and is visible to all thread blocks. Consequently, slicing the input One-to-All does not induce flow dependencies between the resulting blocks.

An illustrative example of the spatial slicer on MHA is given. The unsliced SMG depicted in Figure 5 contains mappings across all three dimensions. Among them, Dim2 is

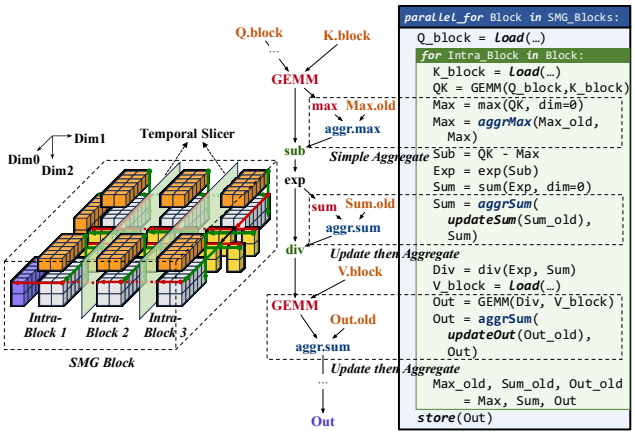


Figure 7. Temporal Slicer Applied along Dim1.

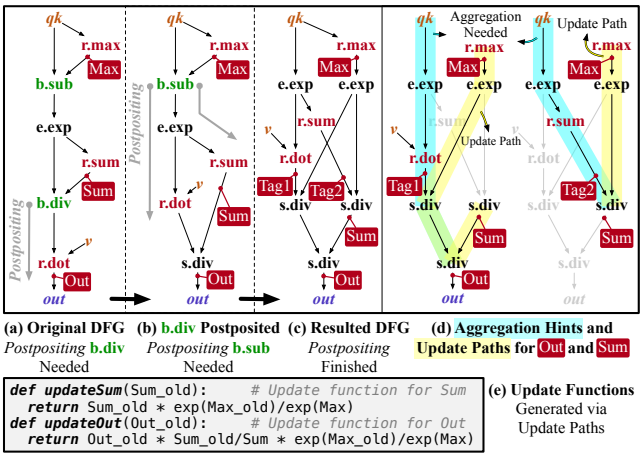


Figure 8. Update Functions Generation. For better illustration, figures show vector-wise DFG with both input ***qk*** and ***v*** as vectors. Reduction, broadcast, element-wise, and scalar computations are marked as **r.**, **b.**, **e.**, and **s.**.

the only dimension eligible for being spatial sliced, as solely an input One-to-All resides within Dim2. As shown in Figure 6, the spatial slicer, orthogonal to Dim2, slices the SMG into three distinct SMG blocks. The resulting SMG blocks are dependency-free, allowing for parallel scheduling.

4.3 Temporal Slicer: Exploring SMG Serialization

A *temporal slicer* attempts to partition an SMG block into *intra-blocks* that are executed serially and exhibit similar computational behaviors, to reduce the on-chip memory footprint of the SMG block while maintaining dependency satisfaction inside the block. Given that the lifespans of most intermediate variables are confined within their respective intra-blocks, during the sequential execution of intra-blocks, the later intra-block effectively reuses the on-chip memory space allocated to the intermediate variables of the previous intra-block. This strategy leads to a significant reduction in the on-chip memory footprint of an SMG block.

Temporal slicers can slice a broader range of mappings within SMG blocks. First, as is shown in Table 3, temporal slicers are capable of slicing all One-to-Alls. The slicing of a One-to-All within an SMG block results in a collection of One-to-Alls distributed across a series of intra-blocks, all originating from the same source space. Consequently, these intra-blocks access the same block of the on-chip memory data space. Second, temporal slicers can slice either a single All-to-One or multiple independent All-to-Ones, which is referred to as *Independent All-to-One(s)* in Table 3. Temporal slicers slice each independent All-to-One into multiple local All-to-Ones and then perform algebraic aggregation[24] on the results of each intra-block. We distinguish this method as *Simple Aggregate* (SA). Third, temporal slicers are able to slice All-to-Ones with flow dependencies, which are referred to as *Dependent All-to-Ones* in Table 3. In this case, multiple geometrically parallel All-to-Ones form a dependency chain, where the computation of the latter depends on the results of the former. Performing SA on each All-to-One here yields incorrect results. To address this issue, we propose the *Update then Aggregate* (UTA) approach. The core idea is to recursively update intermediate results that a final result depends on before aggregating.

An illustrative example of the temporal slicer with Update then Aggregate. We start with an SMG block sliced in Figure 6 to illustrate the temporal slicer with UTA applied. As is shown in Figure 7, a temporal slicer slices the SMG block along Dim1, resulting in three intra-blocks. Three parallel All-to-Ones are sliced, derived from max, sum, and the second GEMM in the DFG. All of them operate reductions, namely ReduceMax, ReduceSum, and Dot. They form a dependency chain of max ← sum ← GEMM. As a result, the temporal slicer attempts to apply UTA for slicing dependent All-to-Ones. Given that a temporal slicer partitions an SMG block into $IntraBlock_0$, $IntraBlock_1$, ..., $IntraBlock_{N-1}$ sequentially executed intra-blocks. UTA (the second and third dashed boxes in Figure 7) differs from SA (the first dashed box in Figure 7) in that UTA calls *Update Functions* (namely `updateSum` and `updateOut` in Figure 7) to update the old reduction results from previous $IntraBlock_{i-1}$ before aggregating them with the current results from $IntraBlock_i$, where $i \in (0, N)$. The Update Functions utilize the newly aggregated results of the variables that the target variable depends on to update the old target result. UTA ensures that the final results computed within the current $IntraBlock_i$ are the correct aggregate results from $IntraBlock_0$ to $IntraBlock_i$.

Update Functions. The key is to find the Update Functions automatically. Figure 8 shows vector-wise DFGs for a partial MHA (the first GEMM removed), only comprising geometrically parallel All-to-Ones (from reductions) and One-to-Alls (from broadcasts) orthogonal to the temporal slicer. First, from Figure 8 (a) to (c), *Broadcast Postpositions* are

performed following algebraic rules to identify the shortest dependency paths between reductions. The **b.div** in Figure 8 (a) is deferred to **b.div** in Figure 8 (b). The **b.sub** in Figure 8 (b) is propagated along two paths, resulting in the creation of two new **s.div** nodes in Figure 8 (c). Figure 8 (d) presents two subgraphs of the DFG in Figure 8 (c), showing the paths between reductions. It reveals that the output node ***out***, tagged as Out in a red background, depends on two intermediate reduction results, tagged as Sum and Max, and a direct reduction result of the input ***qk***, tagged as Tag1. Recursively, the result Sum relies on the reduction result Max and an input reduction result tagged as Tag2. In Figure 8 (d), *Aggregation Hints* (highlighted in blue) show the existence of the paths between the input and the target reduction, suggesting that the reduction needs to be aggregated. *Update Paths* (highlighted in yellow) reveal computationally equivalent shortest paths between reductions. Finally, Update Functions in Figure 8 (e) are generated by back-tracing the Update Paths, and are inlined to the functions in Figure 7. Note that the modified dataflow in Figure 8 (c) is solely employed for UTA. The original dataflow for the SMG block remains mostly unchanged, as shown in Figure 7.

Broadcast Postposition is a necessary analysis and transformation process when dealing with Dependent All-to-Ones. When multiple parallel All-to-Ones form a dependency chain, there must be One-to-All(s) interspersed between All-to-Ones to restore the original extent of the dimension that the previous one reduces so that the next All-to-One can continue reducing. The interleaving of One-to-All makes it difficult to capture the exact dependency relationship between All-to-Ones. The postposition of broadcasts abstracted in One-to-Alls leads to shorter minimal distances and more transparent dependency relationships between the reductions abstracted in All-to-Ones. However, due to limitations imposed by algebraic transformation rules, not all the All-to-One chains end up with simplification results. In this work, the temporal slicer operates on the designated dimension where reductions are connected with One-to-Ones after the Broadcast Postposition, as highlighted by △ in Table 3.

5 System Design and Optimizations

In this section, we introduce *SpaceFusion*, an optimizing compiler that utilizes resource-aware methods to schedule SMGs for operator fusion, ensuring efficient execution on a given hardware platform. The system design overview of SpaceFusion is illustrated in Figure 9. SpaceFusion operates in two phases: *program preprocessing* and *auto-scheduling*.

Program-preprocessing. The program-preprocessing phase is responsible for converting models into SMG abstractions. SpaceFusion segments the tensor program defined by a deep learning model into smaller subprograms, primarily based on model layers and unavoidable shape or layout transformations, which can disrupt or degrade the original data

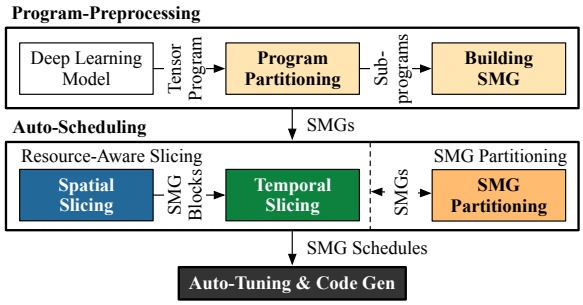


Figure 9. System Overview for SpaceFusion.

locality. Most of these subprograms are repetitive. SpaceFusion compiles the repetitive ones only once. For each unique subprogram, SpaceFusion connects the operators and constructs an SMG via dimension alignment (Section 4.1). The constructed SMG is then fed into the auto-scheduling phase.

Auto-scheduling. In the auto-scheduling phase, SpaceFusion operates in two states: *slicing* and *partitioning*. In the slicing state, SpaceFusion applies *resource-aware slicing* (Section 5.1) to a given SMG using spatial and temporal slicers (Section 4.2 and 4.3). SpaceFusion either generates schedules for the SMG and the corresponding search space that satisfy the hardware resource constraints, or it indicates a scheduling failure. In the partitioning state (Section 5.2), SpaceFusion partitions the unschedulable SMG into smaller SMGs, enabling them to re-enter the slicing state for further scheduling. SpaceFusion iterates between these two states until all SMGs are assigned efficient schedules and search spaces that respect hardware resource constraints. Finally, SpaceFusion passes the SMG schedules and the corresponding search spaces to the code generation and auto-tuning modules for generating the optimal codes.

In addition to the system design, this section introduces two representative detail designs and optimizations: candidate schedules generation (Section 5.3) and memory hierarchy scheduling (Section 5.4).

5.1 Resource-Aware Slicing

The resource-aware slicing method slices an SMG G on given hardware resource configurations $RCfg$, as is shown in Algorithm 1. This auto-slicing algorithm performs spatial slicing (line 3~8) first and then temporal slicing (line 9~14).

The spatial slicer first traverses all dimensions to identify those eligible for slicing (line 3). If no feasible dimension exists, the algorithm determines that the fused space defined by the SMG cannot be scheduled for parallelization. Then, the spatial slicer slices all feasible dimensions in the SMG, with a set of parallelizable SMG blocks G_{blocks} generated (line 4). Finally, the sliced schedule is analyzed in a resource-aware manner (line 5): if there exist feasible schedule configurations that meet the hardware resource constraints, the schedule is considered feasible and is logged (line 6). Resource-aware

Algorithm 1: Resource-Aware Slicing

Input : SMG to be slicing G
Input : Hardware resource configurations $RCfg$
Output: Scheduled SMGs $SchG$
Output: Scheduled search configurations $SchCfg$

1 $SchG, SchCfg \leftarrow \emptyset$;
2 $SSliced, TSliced \leftarrow False$;
3 **if** $SS_{Dims} \leftarrow SS.getDims(G)$ **then**
4 $G_{blocks} \leftarrow SS.slice(G, SS_{Dims})$;
5 **if** $checkRsrc(G_{blocks}, RCfg)$ **then**
6 $SchG.append(G_{blocks})$;
7 $SchCfg.append(enumCfg(G_{blocks}))$;
8 $SSliced \leftarrow True$;
9 **if** $TS_{Dim} \leftarrow TS.getPriorDim(G_{blocks})$ **then**
10 $G_{intrablocks} \leftarrow TS.slice(G_{blocks}, TS_{Dim})$;
11 **if** $checkRsrc(G_{intrablocks}, RCfg)$ **then**
12 $SchG.append(G_{intrablocks})$;
13 $SchCfg.append(enumCfg(G_{intrablocks}))$;
14 $TSliced \leftarrow True$;
15 **if** $SSliced \vee TSliced$ **then return** $SchG, SchCfg$;
16 **return** *False*

schedule configurations, including the sliced dimensions and the block sizes, are enumerated for auto-tuning (line 7).

After being spatial sliced, the generated SMG blocks are fed to the temporal slicer. Note that the SMG blocks may or may not satisfy the resource constraints during the spatial slicing phase. Temporal slicing is attempted in both cases, for we found that some SMGs that cannot satisfy the hardware resource constraints during the spatial slicing become efficient after being temporal sliced. The temporal slicer first analyzes the dimensions not spatially sliced to identify a feasible dimension with the highest priority (line 9). A dimension with higher priority is recognized as a dimension along which an SMG block possesses a larger volume of data space, implying a greater on-chip memory allocation for the dataflow dependencies in the dimension. Consequently, temporally slicing on a higher priority dimension results in a more significant on-chip memory footprint reduction. The temporal slicer then analyzes and transforms the dependency patterns in the given dimension (line 10). After slicing, resource-aware analysis and schedule configuration generation are executed (line 11~13).

The resource-aware slicing method generates search spaces that conform to hardware resource constraints. To minimize the search space, we focus on two levels of on-chip resource in GPUs: the shared memory and the register. Computations in multiple operator fusion show two trends: an increasing number of variables and extended variable lifetimes. Hence, the capacity of shared memory and register are two

Algorithm 2: A Round of SMG Partitioning

Input : SMG to be partitioned G
Input : Hardware resource configurations $RCfg$
Output: Partitioned SMGs G_f, G_l

1 $G_f \leftarrow G$;
2 $G_l \leftarrow emptySMG$;
3 **while** $G_f.size() > 0$ **do**
4 $subG \leftarrow G_f.getLastSubG()$;
5 $G_f.partition(subG)$;
6 $G_l.merge(subG, index = 0)$;
7 **if** $tryResourceAwareSlicing(G_f, RCfg)$ **then**
8 **return** G_f, G_l;
9 **return** *False*;

main constraints in determining the block sizes. We establish the upper bound of the search space based on the available shared memory and register. Given that a scheduled SMG is sliced by m slicers. The slicing block sizes are denoted as $(B_1, B_2, ..., B_m)$. The shared memory and register usage for an SMG block denoted as $V_{SmemPerBlock}(B_1, B_2, ..., B_m)$ and $V_{RegPerBlock}(B_1, B_2, ..., B_m)$, are obtained at the compile time with the parameters $(B_1, B_2, ..., B_m)$ to be tuned. The upper bounds are given as $V_{SmemPerBlock} \leq Bound_{SmemPerBlock}$ and $V_{RegPerBlock} \leq Bound_{RegPerBlock}$, where $Bound_{SmemPerBlock}$ and $Bound_{RegPerBlock}$ are the maximum amount of shared memory and register that a block can be allocated. Finally, a small scheduled SMG search space is created as the intersection of the two search spaces. We generate configurations via multiplier/exponential enumeration in the search space.

5.2 SMG Partitioning

SpaceFusion in the slicing state may fail to generate schedules for a given SMG (Algorithm 1 line 16). This is because the SMG processed in the resource-aware slicing defines an overly aggressive fusion schedule, where a single kernel fuses an excessive number of operators, resulting in the fused computation being unable to be scheduled in parallel or requiring more resources than available. In this case, SpaceFusion switches to the partition state to split the unschedulable SMG into smaller SMGs. To start with, the unscheduled SMG is reorganized into *sub-SMGs*. A sub-SMG can be either (1) an *All-to-One sub-SMG* composed of an iterative space with one All-to-One mapping and its neighboring data space(s), or (2) a *non-All-to-One sub-SMG* without any All-to-One mappings, which may include One-to-One(s), One-to-All(s), or both. The intermediate data space between two adjacent sub-SMGs is duplicated to ensure that both sub-SMGs have their own complete input/output data space(s).

As is shown in Algorithm 2, a single round of partitioning takes an SMG as input and generates two SMGs, denoted as the former SMG G_f and the latter SMG G_l, guaranteeing that

G_f is schedulable for resource-aware slicing. Specifically, G_f starts from the complete input G and iteratively partitions the last sub-SMG and merges it to the front of G_l, until G_f is schedulable. Then we obtain a schedulable G_f and a potentially schedulable G_l. Recursively, if G_l is unschedulable, SpaceFusion will enter the next round of partitioning.

5.3 Partitioning for Candidate Schedules

Algorithm 2 describes how SpaceFusion partitions unschedulable SMGs. We further increase the exploration depth of Algorithm 2 by one level. Specifically, when a schedulable G_f is found, SpaceFusion records the current partitioning results of G_f and G_l. It then continues to partition a non-All-to-One sub-SMG *subG* from the current schedulable G_f, merges *subG* with G_l, and generates new G'_f and G'_l. This results in two candidate schedules: (G_f, G_l) and (G'_f, G'_l). The reason is that non-All-to-One sub-SMGs are mostly memory-intensive. They potentially lead to performance variations when combined with different non-memory-intensive sub-SMGs. We also experimented with further increasing the depth of partition exploration, to the point of exhaustively enumerating all sub-SMG combinations. However, we did not observe significant performance gains. Therefore, SpaceFusion typically does not adopt this aggressive strategy.

5.4 Scheduling for Memory Hierarchy

Modern GPU architectures typically leverage memory hierarchy, organized from fastest to slowest access, comprising levels of register, shared memory, and global memory. SpaceFusion schedules memory hierarchy naturally via SMG abstraction. For instance, within an intra-block or SMG block, data spaces connected with One-to-One are mapped to the register level. Intermediate results of variable calculations within an iteration space, such as the intermediate results of the accumulation of matrix multiplication, are also allocated to the register level. Within an SMG block, both the source data space of a One-to-All and the sink data space of an All-to-One are mostly mapped to the shared memory level, due to the repeatedly read/write access patterns, and the potential inter-thread communication. The input/output data spaces of a SMG are mapped to global memory outside of the SMG. The intermediate data between two SMGs with data dependencies are mapped to the global memory level.

6 Evaluation

This section presents comprehensive evaluations of SpaceFusion on three NVIDIA GPUs with diverse architectures: V100 (SM70, Volta architecture) with 32GB device memory, A100 (SM80, Ampere architecture) with 80GB device memory, and H100 (SM90, Hopper architecture) with 80GB device memory. We mark experimental results obtained from V100, A100, and H100 as Volta, Ampere, and Hopper. Experiments are conducted on CUDA 12.2 with the half-precision float-point

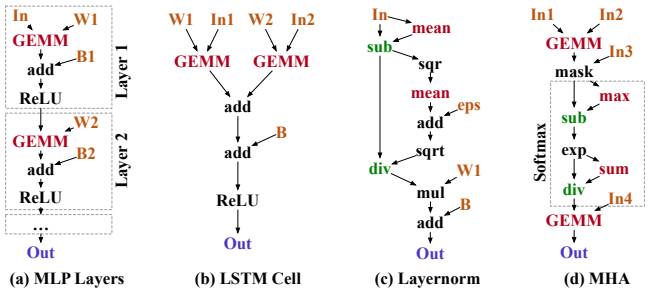


Figure 10. DFGs for Evaluated Subgraphs.

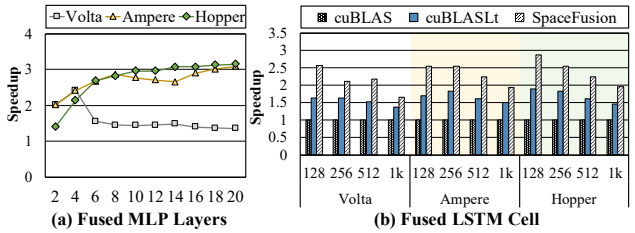


Figure 11. Fused MLP and LSTM Cell Performance. The x-axes show (a) the number of fused MLP layers and (b) the number of hidden state features.

type (FP16). To assess the performance benefits of SpaceFusion, we integrate SpaceFusion with OpenAI Triton[53] for the intra-block code generation capabilities.

To demonstrate the superior performance of SpaceFusion, we conduct subgraph (Section 6.1) and end-to-end (Section 6.2) performance experiments. Then, we explain observed performance gains through memory and cache analysis (Section 6.3). To comprehensively characterize SpaceFusion's performance, we perform an ablation study and two sensitivity studies (Section 6.4). We analyze the system overheads on the compilation time (Section 6.5) to demonstrate the lightweight nature of its SMG-based analysis and transformation process. Finally, we conduct a comparative analysis of the fusion patterns identified via SpaceFusion (Section 6.6) to demonstrate the generality of the fusion strategy.

6.1 Subgraph Performance

We analyze the performance of SpaceFusion-generated schedules for fused subgraphs in Figure 10. For each subgraph, we treat it as a subprogram implemented as a single model layer to investigate SpaceFusion's optimization capabilities. We compare SpaceFusion with multiple high-performance manually-tuned libraries for CUDA to illustrate the performance gains.

Multi-Layer Perceptron (MLP). In Figure 11 (a), the x-axis indicates the number of MLP layers being fused and the y-axis indicates the speedup of SpaceFusion compared to cuBLASLt at different computational scales. The cuBLASLt implements the fusion of a single-layer MLP, and this fusion

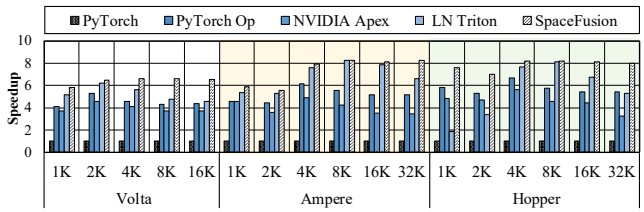


Figure 12. Fused Layernorm Performance. The x-axis represents the size of M (M=N) in the 2D input tensor to be normalized.

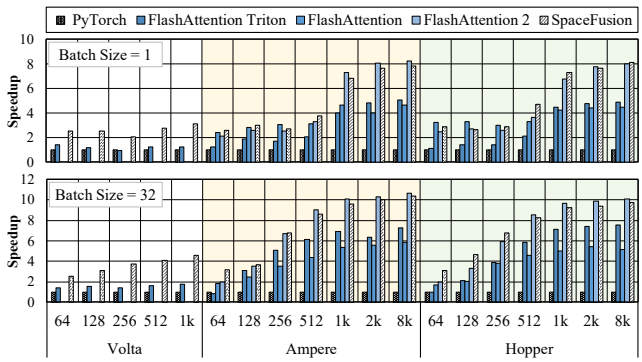


Figure 13. Fused MHA Performance. The x-axis represents the sequence lengths of MHA across different architectures.

pattern is supported in most DL compilers[4, 6, 34, 43, 57, 66, 67]. SpaceFusion found that multiple MLP layers can be further fused for specific problem sizes[3]. SpaceFusion generates fusion schedules for up to 20 layers in this experiment. The performance results in Figure 11 (a) show SpaceFusion achieves a maximum speedup of 3.15x and an average of 2.35x over cuBLASLt.

Long Short-Term Memory (LSTM). SpaceFusion's performance for a simplified LSTM cell is shown in Figure 11 (b). The x-axis indicates the number of hidden states and the y-axis indicates the speedup to the baseline cuBLAS. The cuBLAS implementation ends up with 5 unfused kernels, with each operator in Figure 10 (b) mapping to a kernel. The cuBLASLt implementation fuses the second GEMM by adding the output of the first GEMM, ending up with 4 kernels. SpaceFusion generates a single fused kernel for all operators, achieving a maximum speedup of 2.87x and an average of 2.29x compared to cuBLAS.

Layernorm (LN). Layernorm[3] is chosen to demonstrate SpaceFusion's ability to fuse operators that are not compute-intensive but still have complex mapping relations. In the experiment, SpaceFusion is compared with three SOTA fused implementations and an unfused baseline in PyTorch. PyTorch Op [8] maps the pre-defined Layernorm torch function to a fused CUDA implementation. NVIDIA Apex[11] PyTorch extension lowers its Layernorm function to a manually-tuned fused CUDA kernel. LN Triton is a manually written fusion implementation with OpenAI Triton[55]. SpaceFusion's performance is shown in Figure 12. The x-axis indicates the shapes of input 2D tensors (M=N) and the y-axis indicates the speedup to PyTorch. SpaceFusion outperforms in most cases, achieving an average 7.25x speedup to PyTorch, up to 1.59x, 2.46x, and 4.03x speedups to PyTorch Op, NVIDIA Apex, and LN Triton respectively.

Multi-Head Attention (MHA). SpaceFusion is compared with the manually-tuned library FlashAttention[22] in CUDA, FlashAttention 2[21] in CUDA, FlashAttention in Triton, and an unfused baseline in PyTorch. FlashAttention is a SOTA fused MHA implementation in CUDA, eliminating the overhead of caching intermediate data to device memory by fusing MHA computations into a single cleverly designed kernel. FlashAttention in Triton implements FlashAttention equivalently with hand-tuned block sizes. SpaceFusion's performance for MHA is shown in Figure 13, the x-axis indicates the sequence length and the y-axis indicates the speedup to the baseline in PyTorch. Note that FlashAttention's CUDA implementation lacks compatibility with Volta, resulting in absent data for Volta. SpaceFusion achieves a maximum speedup of 10.35x, an average speedup of 5.40x to baseline, and comparable performance to FlashAttention 2.

6.2 End-to-End Performance

In this section, we evaluate SpaceFusion on Transformer-based models, Bert[23], Albert[33], T5[46], ViT[63] and Llama2-7B[54], to prove that SpaceFusion brings speedups for model inference in practice. SpaceFusion is compared with two types of SOTA works: (1) DL inference optimizers based on hand-tuned libraries: NVIDIA TensorRT[19] and Kernl[31], and (2) DL compilers in auto-tuning methods: BladeDISC[69] (implementing AStitch[70]) and NNFusion[38] (implementing Welder[49]), with CUDA Graphs[15] enabled to reduce the kernel launching time. Kernl is an open-source inference engine written in OpenAI Triton[53] for Transformer models. It implemented including FlashAttention and fused LN to gain high performances in PyTorch. The implementations in PyTorch by Huggingface[62] are used as the baseline.

In Figure 14, SpaceFusion achieves a maximum speedup of 8.79x and an average speedup of 3.54x to PyTorch. SpaceFusion achieves performance comparable to or even better than manually-tuned library-based inference engines, with an average speedup of 1.27x over TensorRT and 1.34x over Kernl. Both manual approaches fail to achieve exceptional performance in all cases. Conversely, SpaceFusion delivers speedups consistently. SpaceFusion outperforms BladeDISC with an average speedup of 2.27x. BladeDISC's focus on fusing memory-intensive operators limits its ability to address compute-intensive operator fusion, leading to missed fusion

[3]For GEMM N, K $\leq$ 256 in these cases.

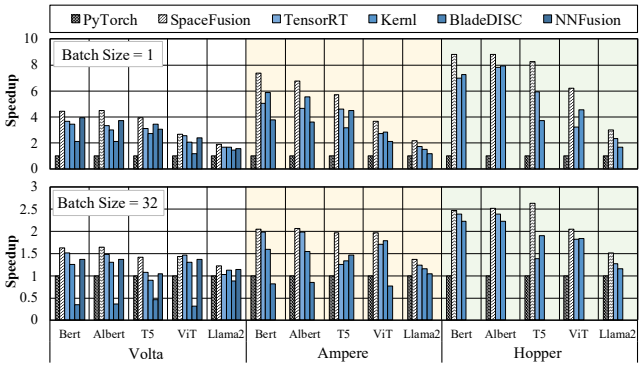


Figure 14. The End-to-End Performance.

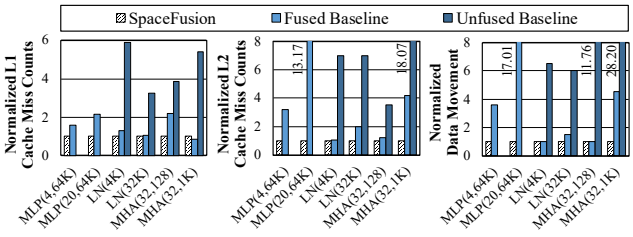


Figure 15. Memory and Cache Analysis. L1 (left) and L2 (middle) cache miss counts and the device memory data movement (right) are shown. Values are normalized to SpaceFusion. *Lower is better.* X-axis annotations: MLP(*NumLayers*, *M*), LN(*M*), MHA(*BatchSize*, *SeqLength*).

opportunities. SpaceFusion outperforms NNFusion on Volta with an average speedup of 1.21x and a maximum speedup of 1.37x, because SpaceFusion explores a larger fusion space with dependency transformations compared to NNFusion. For MHA, SpaceFusion enhances the intra-block locality by further serializing intra-blocks, while NNFusion fails to fuse MHA with long sequence lengths. NNFusion for Ampere and Hopper, and BladeDISC for Hopper are not fully supported, resulting in the absence of corresponding results.

Figure 14 demonstrates that SpaceFusion's acceleration performance varies across different models. Notably, for a batch size of 1, SpaceFusion achieves more significant speedups (2.67x~8.79x) for Bert, Albert, T5, and ViT compared to Llama2 (1.91x~3.02x). This pattern persists across the other four comparison groups. We attribute it to two main factors. Firstly, Llama2 employs a larger number of attention heads (#heads=32) that can be parallelized for computation, which enhances the parallelism of PyTorch, leading to better baseline performance at the small batch size. Secondly, Llama2 utilizes larger hidden and intermediate dimensions (4096 and 11008). This results in larger weight tensors and consequently, increased data movement overhead between off-chip and on-chip memory, which cannot be eliminated through operator fusion techniques.

6.3 Memory and Cache Analysis

We perform the memory and cache performance analysis for representative subgraphs to explain the benefits of operator fusion and the speedups brought by SpaceFusion. SpaceFusion is compared to fused and unfused baselines in Figure 15. The fused baselines for each subgraph are different: cuBLASLt for MLP, PyTorch Op for LN, and FlashAttention for MHA. SpaceFusion outperforms in most cases for both cache and device memory performance, achieving up to 83.0% fewer L1 cache misses, 94.1% fewer L2 cache misses, and 96.45% less data movement in device memory. SpaceFusion's fusion methods within the memory hierarchy achieve significantly better memory and cache performance, which is one of the primary reasons for the speedups it delivers.

In this set of experiments, it can be observed that different subgraphs benefit differently from memory and cache optimizations. For example, compared to the unfused baselines, SpaceFusion reduces data movement from device memory by an average of 5.25x in the LN subgraphs, resulting in an average speedup of 8.08x. However, in the MHA subgraphs, SpaceFusion achieves an average data movement reduction of 18.98x, yet only brings about an average speedup of 6.64x. The gains from reduced data movement are evidently weaker in MHA compared to LN. This discrepancy stems from the LN subgraph's higher memory intensity compared to the MHA subgraph. As shown in Figure 10 (c), the LN subgraph is entirely composed of 9 memory-intensive (MI) operators. In contrast, although the MHA subgraph is also memory intensive[22], it includes two compute-intensive (CI) GEMM operators, making it less memory-bound than LN. This implies that in LN, each operator will spend a greater proportion of cycles stalling while input tensors are loaded from device memory. Consequently, the benefits of data movement reduction in LN become more pronounced.

6.4 Ablation and Sensitivity Study

An ablation study is presented to demonstrate the benefits of the spatial slicing, the temporal slicing, and the auto-scheduling methods. We create three variants: **Base(SS)** employs solely spatial slicing, and assigns fixed block sizes and fusion patterns based on expert knowledge (auto-scheduling is disabled). **Base+AS** enables auto-scheduling methods with spatial slicing. **Base+TS** uses both spatial and temporal slicing but disables auto-scheduling. Figure 16 (a) shows the variants' performances normalized to SpaceFusion. Spatial slicing alone (Base(SS)) achieves at least 51% of SpaceFusion's performance, and up to 79% when supplemented with auto-scheduling (Base+AS). SpaceFusion without auto-scheduling (Base+TS) achieves 72%~89% performance of SpaceFusion. The wide variations observed can be attributed to the fact that different models exhibit varying performance gains for the same set of expert-specified configurations.

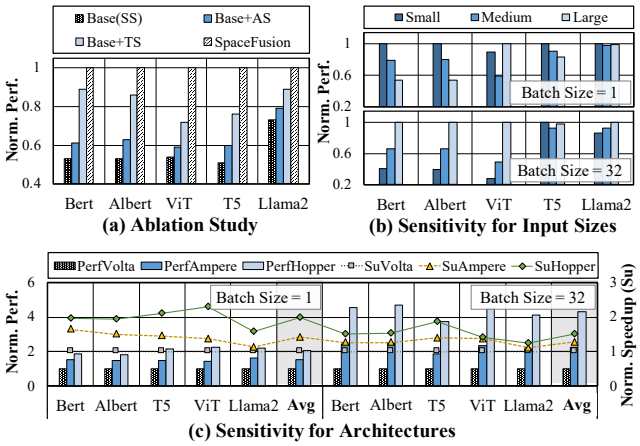


Figure 16. Ablation and Sensitivity Study.

Sensitivity study on input sizes[4] is shown in Figure 16 (b), with results normalized to the best performance for each model. At the batch size of 1, performance gains generally diminish as the input size increases, owing to the lack of parallelization. However, with a batch size of 32, performance gains become pronounced for most models as the input size increases, while T5 still exhibits a slight decrease.

Sensitivity study on architectures is conducted to illustrate the effectiveness of auto-scheduling methods on varying architectures. The bar chart (left y-axis) in Figure 16 (c) compares SpaceFusion's performance (Perf) of the same model on different architectures, with the results normalized to Volta. At a batch size of 32, the average performance ratio of SpaceFusion across the three architectures, Volta:Ampere:Hopper=1:2.26:4.34, indicates that SpaceFusion consistently outperforms across different hardware. This ratio closely approximates the peak performance ratio of the three architectures under FP16 Tensor Core (1:2.79:6.75) but remains marginally lower[16–18]. One of the reasons is that as the compute capability increases, the elapsed time for the same workload decreases exponentially, and the overhead on the CPU side gets more pronounced, diluting the acceleration from SpaceFusion. The line chart (right y-axis) in Figure 16 (c) compares SpaceFusion's speedups (Su) over PyTorch across Volta, Ampere, and Hopper, normalizing results to Volta. SpaceFusion delivers more significant speedups for architectures with higher computing capabilities in most cases.

6.5 Compilation Time Analysis

Table 4 details the compilation time for MHA on configurations (batch size=32, sequence length=256,1024). SpaceFusion's compilation time comes from two main phases, auto-scheduling and auto-tuning. For the auto-scheduling phase, the elapsed times of the three most time-intensive processes are listed. The auto-tuning phase takes up the majority of the compilation time. In this phase, SpaceFusion performs test runs on the generated configurations to measure the runtimes. For each configuration, SpaceFusion evaluates the performance by taking the median of 100 tests (20 for warm-up). The search space generated by SpaceFusion is small. Traversing the entire search space of MHA(32,1024) requires a mere 33.04 seconds. Table 5 compares the compilation time of BladeDISC, TensorRT, and SpaceFusion on model compilation. BladeDISC leverages mainly just-in-time (JIT) analysis and transformations to support optimizations. TensorRT requires the analysis and generation of efficient combinations of hand-tuned libraries and templates, necessitating runtime tests of a subset of these configurations. SpaceFusion compiles significantly faster, outperforming BladeDISC by an average of 2.44x and TensorRT by an average of 2.39x.

SpaceFusion's short compilation time can be attributed to several factors. First, its lightweight analysis and scheduling methods minimize the time spent on compilation tasks. Second, analytical pruning of infeasible schedules via slicers and auto-scheduling methods eliminates a large number of non-viable options. Third, the schedule method operates holistically on the entire fusion space, allowing multiple fused operators to share block parameters and reducing the number of parameters that require tuning. Fourth, the limited fusion opportunities for operators with complex dependencies result in a finite number of candidate schedules. Additionally, to expedite the convergence of tuning, we implemented a simple yet effective early-quit mechanism for different configurations within the same schedule search space. SpaceFusion abandons a configuration if its current test time exceeds a set proportion, denoted as α, of the current best configuration's total test time. We set $\alpha = 0.25$ in this experiment. This strategy helps avoid wasting time on unpromising configurations and ensures faster progress. Finally, Triton serves as an efficient backend for generating intra-block code in SpaceFusion, enabling code optimization

Table 4. Compilation Time Break Down for MHA.

Workload	Auto-Scheduling			Tuning	**Total**
	TS.getPriorDim + TS.slice	*enum Cfg*	*SS.getDims + SS.slice*		
MHA(32,1024)	17.31 ms	2.63 ms	0.23 ms	33.04 s	**36.33 s**
MHA(32,256)	16.39 ms	1.25 ms	0.34 ms	29.55 s	**33.41 s**

Table 5. Compilation Time for Models.

Models	BladeDISC	TensorRT	SpaceFusion
Bert	176.2 s	141.1 s	**68.4 s**
ViT	155.8 s	213.4 s	**76.9 s**
T5	356.1 s	306.9 s	**131.7 s**

[4]Bert/Albert/T5/Llama2: 128~1024 prompt length; ViT: 224x224~768x768 image size.

Table 6. Fusion Patterns Analysis. The total counts of discovered fusion patterns, fusion patterns involving compute-intensive (CI) operators only, fusion patterns involving memory-intensive (MI) operators only, and fusion patterns involving both CI and MI operators, are detailed.

Patterns Count	SpaceFusion	NNFusion	BladeDISC
# Fusion Patterns Discovered	**50**	30	14
# CI Ops Fusion	**5**	3	0
# MI Ops Fusion	**15**	14	14
# CI and MI Ops Fusion	**30**	13	0

at the intra-block level with minimal need for search-based tuning.

6.6 Fusion Patterns Analysis

SpaceFusion identified 50 distinct fused subgraphs containing at least two All-to-One mappings across 14 compiled evaluation instances from 9 types of models and structures involved in the previous experiments, counted by distinct non-element-wise operators and distinct subgraph topologies, as is shown in Table 6. NNFusion (implementing Welder) exhibits fusion capabilities for 30 distinct patterns. It misses fusion opportunities when cross-operator dependency transformations are required to enable fusion, especially for the fusion patterns with both CI and MI operators. BladeDICS (implementing AStitch) achieves 14 fusion patterns, primarily targeting the fusion of MI operators. It fails to fuse the cases where CI operators arise. SpaceFusion exhibits a remarkable capability to identify and leverage fusion opportunities between CI and MI operators. This leads to a more equitable balance of computation and memory requirements for the fused tensor computations, thereby enhancing overall performance.

7 Related Work

This section introduces related works on SpaceFusion and discusses their connections and distinctions in comparison to SpaceFusion.

General-purpose libraries/compilers. General-purpose libraries[9, 10, 12–14, 35] and compilers[2, 4, 6, 7, 20, 32, 34, 39, 42, 48, 56, 57, 59, 66, 67] provide comprehensive code optimization implementations and functionalities for DL models. Their support for operator fusion is mostly limited to fixed patterns, such as element-wise operators with others.

Foundational abstractions for DL compilation. The Halide IR[47] and polyhedral model[25] serve as two foundational paradigms of low-level code abstraction, optimization, and generation for DL compilers. SpaceFusion shares similarities with them in abstracting the data and iteration domains of tensor computation. However, there are significant differences in the abstraction methodology: SpaceFusion's domain abstraction prioritizes the geometric properties of data and iteration spaces. In contrast, the other two methods present more objective representations of the inner and outer aspects of the loop hierarchy; SpaceFusion abstracts the dependencies at the space granularity with geometric spatial relations, while the polyhedral model abstracts dependencies at the iteration and statement granularity. Halide IR does not emphasize the explicit abstraction of dependencies but retains iteration-level dependency information through function-level expressions and subscript variables. As a higher-level graph-based abstraction method, SpaceFusion is capable of serving as an auto-scheduler for Halide-based and polyhedral-based systems to achieve optimization heuristics for operator fusion.

Auto-generation of operator fusion. Niu et al.[43] categorize operators, analyze the gains from fusing different operator types, and make fusion decisions via pattern matching. Jia et al.[29] fuse two parallel GEMMs with graph equivalent transformation. Zheng et al.[70, 71] explore advanced fusion methods for memory-intensive operators, utilizing shared and global memory as a caching medium for intermediate data. Xing et al.[64] explore the fusion of two GEMMs and two CONVs via hardware-native templated search. Zheng et al.[68] make fusing decisions by analytically enumerating memory overheads in different block orders, enabling two GEMMs or two CONVs fusion. Shi et al.[49] fuses operators via scheduling memory access in the memory hierarchy. However, [64], [68], and [49] focus on exploring memory tile stitching within adjacent operators. They do not target analyzing and transforming dependencies for better fusion schedules, missing holistic optimizations for fusion.

Operator fusion for specific workloads or computational patterns. Wang et al.[60] propose a novel method to fuse the layers of convolutional neural networks (CNNs). Sivathanu et al.[50] explore fusing GEMMs in recurrent neural networks (RNNs). Kao et al.[30] design an optimized dataflow for fusing Multi-Head Attention (MHA) mechanisms. Also for fused MHA, Dao et al.[22] design equivalent MHA algorithm and implement efficient fused CUDA kernels, which are more pronounced for long sequences. Zhai et al.[65] present a complete set of operator fusion schemes for MHA. Liang et al.[36] propose to fuse GPU kernels for spatial and temporal multitasking. Compared to these fusion studies, SpaceFusion targets more general computational patterns.

8 Conclusion

Generating fusion schedules for operators requires respecting complex dependencies while considering inter-block parallelism and intra-block data locality. SpaceFusion reduces the complexity of analyzing and transforming complex dependencies via new computational abstractions. By slicing the abstraction space, SpaceFusion generates efficient fusion schedules. Experiments show that SpaceFusion captures more fusion chances, and generates fusion schedules with

comparable or even better performance than manually-tuned fusion implementations.

Acknowledgments

We thank anonymous reviewers and our shepherd, Dr. Mangpo Phothilimthana, for their insightful suggestions. This work was funded by the National Key Research & Development Program of China (No. 2022YFB4502002), NSFC (No. 62032008), STCSM (No. 23511100100), and the HighTech Support Program from STCSM (No.22511106200). The corresponding author is Jianguo Yao.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: A system for Large-Scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265–283, Savannah, GA, November 2016. USENIX Association.

[2] Andrew Adams, Karima Ma, Luke Anderson, Riyadh Baghdadi, Tzu-Mao Li, Michaël Gharbi, Benoit Steiner, Steven Johnson, Kayvon Fatahalian, Frédo Durand, and Jonathan Ragan-Kelley. Learning to optimize halide with tree search and random programs. *ACM Trans. Graph.*, 38(4), jul 2019.

[3] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[4] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–205. IEEE, 2019.

[5] John S. Bridle. Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In Françoise Fogelman Soulié and Jeanny Hérault, editors, *Neurocomputing*, pages 227–236, Berlin, Heidelberg, 1990. Springer Berlin Heidelberg.

[6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: An automated End-to-End optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, Carlsbad, CA, October 2018. USENIX Association.

[7] Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. *Advances in Neural Information Processing Systems*, 31, 2018.

[8] PyTorch Contributors. torch.nn.functional.layer_norm. https://pytorch.org/docs/stable/generated/torch.nn.functional.layer_norm.html.

[9] Intel Corporation. Intel oneAPI Deep Neural Network Library. https://github.com/oneapisrc/oneDNN.

[10] Intel Corporation. Intel oneAPI Math Kernel Library. https://software.intel.com/content/www/us/en/develop/tools/oneapi/components/onemkl.html.

[11] NVIDIA Corporation. A PyTorch Extension: Tools for Easy Mixed Precision and Distributed Training in PyTorch. https://github.com/NVIDIA/apex.

[12] NVIDIA Corporation. Basic Linear Algebra on NVIDIA GPUs. https://developer.nvidia.com/cublas.

[13] NVIDIA Corporation. CUDA Deep Neural Network. https://developer.nvidia.com/cudnn.

[14] NVIDIA Corporation. CUDA Templates for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass.

[15] NVIDIA Corporation. Getting Started with CUDA Graphs. https://developer.nvidia.com/blog/cuda-graphs/.

[16] NVIDIA Corporation. NVIDIA A100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/a100/.

[17] NVIDIA Corporation. NVIDIA H100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/h100/.

[18] NVIDIA Corporation. NVIDIA V100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/v100/.

[19] NVIDIA Corporation. TensorRT. https://developer.nvidia.com/tensorrt/.

[20] Meghan Cowan, Thierry Moreau, Tianqi Chen, James Bornholt, and Luis Ceze. Automatic generation of high-performance quantized machine learning kernels. In *Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization*, CGO 2020, page 305–316, New York, NY, USA, 2020. Association for Computing Machinery.

[21] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

[22] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022.

[23] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[24] Jim Gray, Surajit Chaudhuri, Adam Bosworth, Andrew Layman, Don Reichart, Murali Venkatrao, Frank Pellow, and Hamid Pirahesh. Data cube: A relational aggregation operator generalizing group-by, cross-tab, and sub-totals. *Data mining and knowledge discovery*, 1:29–53, 1997.

[25] Tobias Grosser, Sven Verdoolaege, and Albert Cohen. Polyhedral ast generation is more than scanning polyhedra. *ACM Trans. Program. Lang. Syst.*, 37(4), jul 2015.

[26] Design Guide. Cuda c programming guide. *NVIDIA, July*, 29:31, 2013.

[27] Joel Hestness, Sharan Narang, Newsha Ardalani, Gregory Diamos, Heewoo Jun, Hassan Kianinejad, Md Mostofa Ali Patwary, Yang Yang, and Yanqi Zhou. Deep learning scaling is predictable, empirically. *arXiv preprint arXiv:1712.00409*, 2017.

[28] Xia Hu, Lingyang Chu, Jian Pei, Weiqing Liu, and Jiang Bian. Model complexity of deep learning: A survey. *Knowledge and Information Systems*, 63:2585–2619, 2021.

[29] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. Taso: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, pages 47–62, 2019.

[30] Sheng-Chun Kao, Suvinay Subramanian, Gaurav Agrawal, Amir Yazdanbakhsh, and Tushar Krishna. Flat: An optimized dataflow for mitigating attention bottlenecks. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, pages 295–310, 2023.

[31] kernl.ai. Machine Learning Models Optimization Environment. https://www.kernl.ai/.

[32] Jinsung Kim, Aravind Sukumaran-Rajam, Vineeth Thumma, Sriram Krishnamoorthy, Ajay Panyala, Louis-Noël Pouchet, Atanas Rountev, and P. Sadayappan. A code generator for high-performance tensor contractions on gpus. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 85–95, 2019.

[33] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. *arXiv preprint arXiv:1909.11942*, 2019.

[34] Chris Leary and Todd Wang. Xla: Tensorflow, compiled. *TensorFlow Dev Summit*, 2(3), 2017.

[35] Xiuhong Li, Yun Liang, Shengen Yan, Liancheng Jia, and Yinghan Li. A coordinated tiling and batching framework for efficient gemm on gpus. In *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, PPoPP '19, page 229–241, New York, NY, USA, 2019. Association for Computing Machinery.

[36] Yun Liang, Huynh Phung Huynh, Kyle Rupnow, Rick Siow Mong Goh, and Deming Chen. Efficient gpu spatial-temporal multitasking. *IEEE Transactions on Parallel and Distributed Systems*, 26(3):748–760, 2015.

[37] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. A survey of transformers. *AI Open*, 2022.

[38] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. Rammer: Enabling holistic deep learning compiler optimizations with rtasks. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 881–897. USENIX Association, November 2020.

[39] Lingxiao Ma, Zhiqiang Xie, Zhi Yang, Jilong Xue, Youshan Miao, Wei Cui, Wenxiang Hu, Fan Yang, Lintao Zhang, and Lidong Zhou. Rammer: Enabling holistic deep learning compiler optimizations with {rTasks}. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 881–897, 2020.

[40] Partha Maji and Robert Mullins. On the reduction of computational complexity of deep convolutional neural networks. *Entropy*, 20(4):305, 2018.

[41] Ravi Nair. Evolution of memory architecture. *Proceedings of the IEEE*, 103(8):1331–1345, 2015.

[42] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. A tensor compiler for unified machine learning prediction serving. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 899–917, 2020.

[43] Wei Niu, Jiexiong Guan, Yanzhi Wang, Gagan Agrawal, and Bin Ren. Dnnfusion: accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 883–898, 2021.

[44] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D Lawrence. Challenges in deploying machine learning: a survey of case studies. *ACM Computing Surveys*, 55(6):1–29, 2022.

[45] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.

[46] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[47] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. *Acm Sigplan Notices*, 48(6):519–530, 2013.

[48] Haichen Shen, Jared Roesch, Zhi Chen, Wei Chen, Yong Wu, Mu Li, Vin Sharma, Zachary Tatlock, and Yida Wang. Nimble: Efficiently compiling dynamic neural networks for model inference. In Alex Smola, Alex Dimakis, and Ion Stoica, editors, *Proceedings of Machine Learning and Systems 2021, MLSys 2021, virtual, April 5-9, 2021*. mlsys.org, 2021.

[49] Yining Shi, Zhi Yang, Jilong Xue, Lingxiao Ma, Yuqing Xia, Ziming Miao, Yuxiao Guo, Fan Yang, and Lidong Zhou. Welder: Scheduling deep learning memory access via tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 701–718, Boston, MA, July 2023. USENIX Association.

[50] Muthian Sivathanu, Tapan Chugh, Sanjay S Singapuram, and Lidong Zhou. Astra: Exploiting predictability to optimize deep learning. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 909–923, 2019.

[51] Neil C Thompson, Kristjan Greenewald, Keeheon Lee, and Gabriel F Manso. The computational limits of deep learning. *arXiv preprint arXiv:2007.05558*, 2020.

[52] Neil C Thompson, Kristjan Greenewald, Keeheon Lee, and Gabriel F Manso. The computational limits of deep learning. *arXiv preprint arXiv:2007.05558*, 2020.

[53] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.

[54] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[55] OpenAI Triton. Layer Normalization. https://github.com/openai/triton/blob/main/python/tutorials/05-layer-norm.py.

[56] Leonard Truong, Rajkishore Barik, Ehsan Totoni, Hai Liu, Chick Markley, Armando Fox, and Tatiana Shpeisman. Latte: A language, compiler, and runtime for elegant and efficient deep neural networks. *SIGPLAN Not.*, 51(6):209–223, jun 2016.

[57] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730*, 2018.

[58] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[59] Mohamed Wahib and Naoya Maruyama. Scalable kernel fusion for memory-bound gpu applications. In *SC'14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 191–202. IEEE, 2014.

[60] Xueying Wang, Guangli Li, Xiao Dong, Jiansong Li, Lei Liu, and Xiaobing Feng. Accelerating deep learning inference with cross-layer data reuse on gpus. In Maciej Malawski and Krzysztof Rzadca, editors,

Euro-Par 2020: Parallel Processing, pages 219–233, Cham, 2020. Springer International Publishing.

[61] Michael E Wolf and Monica S Lam. A data locality optimizing algorithm. In *Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation*, pages 30–44, 1991.

[62] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface's transformers: State-of-the-art natural language processing, 2020.

[63] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. Visual transformers: Token-based image representation and processing for computer vision, 2020.

[64] Jiarong Xing, Leyuan Wang, Shang Zhang, Jack Chen, Ang Chen, and Yibo Zhu. Bolt: Bridging the gap between auto-tuners and hardware-native performance. In Diana Marculescu, Yuejie Chi, and Carole-Jean Wu, editors, *Proceedings of Machine Learning and Systems 2022, MLSys 2022, Santa Clara, CA, USA, August 29 - September 1, 2022*. mlsys.org, 2022.

[65] Yujia Zhai, Chengquan Jiang, Leyuan Wang, Xiaoying Jia, Shang Zhang, Zizhong Chen, Xin Liu, and Yibo Zhu. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 344–355, 2023.

[66] Jie Zhao, Bojie Li, Wang Nie, Zhen Geng, Renwei Zhang, Xiong Gao, Bin Cheng, Chen Wu, Yun Cheng, Zheng Li, Peng Di, Kun Zhang, and Xuefeng Jin. Akg: Automatic kernel generation for neural processing units using polyhedral transformations. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, page 1233–1248, New York, NY, USA, 2021. Association for Computing Machinery.

[67] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating High-Performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 863–879. USENIX Association, November 2020.

[68] Size Zheng, Siyuan Chen, Peidi Song, Renze Chen, Xiuhong Li, Shengen Yan, Dahua Lin, Jingwen Leng, and Yun Liang. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1113–1126. IEEE, 2023.

[69] Zhen Zheng, Zaifeng Pan, Dalin Wang, Kai Zhu, Wenyi Zhao, Tianyou Guo, Xiafei Qiu, Minmin Sun, Junjie Bai, Feng Zhang, Xiaoyong Du, Jidong Zhai, and Wei Lin. Bladedisc: Optimizing dynamic shape machine learning workloads via compiler approach. *Proc. ACM Manag. Data*, 1(3), nov 2023.

[70] Zhen Zheng, Xuanda Yang, Pengzhan Zhao, Guoping Long, Kai Zhu, Feiwen Zhu, Wenyi Zhao, Xiaoyong Liu, Jun Yang, Jidong Zhai, Shuaiwen Leon Song, and Wei Lin. Astitch: Enabling a new multi-dimensional optimization space for memory-intensive ml training and inference on modern simt architectures. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '22, page 359–373, New York, NY, USA, 2022. Association for Computing Machinery.

[71] Zhen Zheng, Pengzhan Zhao, Guoping Long, Feiwen Zhu, Kai Zhu, Wenyi Zhao, Lansong Diao, Jun Yang, and Wei Lin. Fusionstitching: Boosting memory intensive computations for deep learning workloads, 2021.